Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of Electrical & Computer Engineering

Lab Manual

Machine Learning Sessional
ECE 4246

(0.75 Credit)

Lab Manuals have been developed for the requirements of course of ECE 3218

Department of Electrical & Computer Engineering
Faculty of Electrical & Computer Engineering
Rajshahi University of Engineering & Technology
Rajshahi-6204

ECE 4212
Machine Learning Sessional
Credit: 0.75

INDEX

SL No.	Experiment Name
01	Implementation and Performance Analysis of Linear and Logistic Regression Models
02	Implementation and Performance Analysis of classification problem using Support Vector Machines (SVM)
03	Implementation and Performance Analysis of probabilistic classification using Naïve Bayes Algorithm
04	Design and Implementation of Single Layer Perceptron and Multi-Layer Perceptron (MLP)
05	Development of Neuro-Fuzzy Systems for Intelligent Decision Making
06	Optimization of Neural Network Parameters Using Genetic Algorithms (Neuro-Genetic Approach)

Experiment No: 01

Name of the Experiment: Implementation and Performance Analysis of Linear and Logistic Regression Models

THEORY

Regression is a supervised learning technique used to establish a relationship between dependent and independent variables.

Linear Regression

Linear Regression predicts a **continuous output** by fitting a straight line to the data using the least squares method.

$$y = wx + b$$

The parameters www (weight) and b (bias) are learned by minimizing the **Mean Squared Error (MSE)** using gradient descent.

Logistic Regression

Logistic Regression is used for **binary classification**. It applies the sigmoid function to map linear combinations of inputs into probability values.

$$\sigma(z) = \{1\}/\{1 + e^\{-z\} \}$$

The model is trained by minimizing **binary cross-entropy loss**.

REQUIRED APPARATUS

Hardware

- Computer / Laptop

Software

- Python 3.x
- NumPy
- Matplotlib (optional for visualization)

Algorithm

Linear Regression Algorithm

1. Initialize weight and bias.
2. Predict output using linear equation.
3. Compute loss (MSE).
4. Update parameters using gradient descent.
5. Repeat until convergence.

Logistic Regression Algorithm

1. Initialize weight and bias.
2. Apply sigmoid function to linear output.
3. Compute classification loss.
4. Update parameters using gradient descent.
5. Predict class labels using threshold.

Code

Linear Regression (From Scratch)

```
import numpy as np

# PSEUDOCODE:
# 1. Initialize weight and bias
# 2. Repeat for fixed iterations:
#    a. Predict y = wx + b
#    b. Compute loss (MSE)
#    c. Update w and b using gradient descent
# 3. Display final parameters

# Dataset
X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

# Initialize parameters
w = 0.0
b = 0.0
learning_rate = 0.01
epochs = 1000
n = len(X)

# Training loop
for _ in range(epochs):
    y_pred = w * X + b
    dw = (-2/n) * np.sum(X * (y - y_pred))
    db = (-2/n) * np.sum(y - y_pred)
    w -= learning_rate * dw
    b -= learning_rate * db

# Output
```

```
print("Final Weight:", w)
print("Final Bias:", b)
```

Logistic Regression (From Scratch)

```
import numpy as np

# PSEUDOCODE:
# 1. Initialize weight and bias
# 2. Apply sigmoid activation
# 3. Compute binary cross-entropy loss
# 4. Update parameters using gradient descent
# 5. Predict class labels

# Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Dataset (binary classification)
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([0, 0, 0, 1, 1])

# Initialize parameters
w = 0.0
b = 0.0
learning_rate = 0.1
epochs = 1000
n = len(X)

# Training loop
for _ in range(epochs):
    z = w * X.flatten() + b
    y_pred = sigmoid(z)

    dw = (1/n) * np.sum((y_pred - y) * X.flatten())
    db = (1/n) * np.sum(y_pred - y)

    w -= learning_rate * dw
    b -= learning_rate * db

# Prediction
y_class = [1 if i >= 0.5 else 0 for i in y_pred]

print("Predicted Classes:", y_class)
```

RESULTS

- Linear Regression converged to optimal weight and bias values.
- Logistic Regression correctly classified most data points.
- Performance depends on learning rate and number of iterations.

Discussion

This experiment demonstrates how regression models work internally without relying on built-in machine learning libraries. Implementing algorithms from scratch provides better insight into parameter optimization, loss minimization, and convergence behavior. Linear Regression is suitable for continuous prediction, while Logistic Regression is effective for binary classification.

References

Experiment No.: 02

Experiment Name

Implementation and Performance Analysis of Classification Problems Using Support Vector Machines (SVM)

Theory

Support Vector Machine (SVM) is a powerful supervised learning algorithm used for **classification and regression** tasks. SVM aims to find an optimal hyperplane that maximizes the **margin** between two classes.

For a binary classification problem, SVM tries to satisfy:

$$y_i(w \cdot x_i + b) \geq 1$$

where

- www is the weight vector
- bbb is the bias
- $y_i \in \{-1, +1\}$

To handle misclassification, **hinge loss** is used:

$$L = \max(0, 1 - y(w \cdot x + b))$$

The optimization problem is solved using **gradient descent**.

Required Apparatus

Hardware

- Computer / Laptop

Software

- Python 3.x
- NumPy
- Matplotlib (optional)

Algorithm

SVM Algorithm (Linear Kernel)

1. Initialize weight vector and bias.
2. For each epoch:
    - Compute hinge loss.
    - Update weights and bias using gradient descent.
3. Use the trained model to classify test samples.
4. Evaluate performance using accuracy.

Code

Support Vector Machine (From Scratch)

```
import numpy as np

# PSEUDOCODE:
# 1. Initialize weights and bias
# 2. Repeat for fixed epochs:
#    a. For each data point:
#       i. Check margin condition
#       ii. Update weights and bias using hinge loss gradient
# 3. Predict class labels using sign(w.x + b)

# Dataset (linearly separable)
X = np.array([[2, 3], [1, 1], [2, 1], [3, 2], [7, 8], [8, 8], [9, 7],
[8, 6]])
y = np.array([-1, -1, -1, -1, 1, 1, 1, 1])

# Initialize parameters
w = np.zeros(X.shape[1])
b = 0
learning_rate = 0.01
lambda_param = 0.01
epochs = 1000

# Training loop
for _ in range(epochs):
    for idx, x_i in enumerate(X):
        condition = y[idx] * (np.dot(w, x_i) + b) >= 1

        if condition:
            w -= learning_rate * (2 * lambda_param * w)
        else:
            w -= learning_rate * (2 * lambda_param * w - y[idx] * x_i)
            b -= learning_rate * y[idx]

# Prediction
def predict(X):
    return np.sign(np.dot(X, w) + b)

y_pred = predict(X)
```

```
# Accuracy
accuracy = np.mean(y_pred == y)
print("Classification Accuracy:", accuracy)
```

RESULTS

- The SVM classifier successfully separated the two classes using a linear hyperplane.
- High classification accuracy was achieved on linearly separable data.
- The margin maximization principle improved generalization.

DISCUSSION

Support Vector Machine focuses on maximizing the margin between classes, making it robust against overfitting. Implementing SVM from scratch provides insight into hinge loss, regularization, and gradient-based optimization. While linear SVM works well for linearly separable data, kernel-based SVMs are required for non-linear problems.

REFERENCES

Experiment No.: 03

Experiment Name

Implementation and Performance Analysis of Probabilistic Classification Using the Naïve Bayes Algorithm

Theory

Naïve Bayes is a **probabilistic supervised learning algorithm** based on **Bayes' Theorem** with the assumption of **conditional independence** among features.

Bayes' Theorem is expressed as:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

where

- C is the class label
- X is the feature vector

Despite its simplicity, Naïve Bayes performs well in many real-world classification problems, especially when the dimensionality of the input is high.

In this experiment, **Gaussian Naïve Bayes** is implemented, assuming features follow a normal distribution.

Required Apparatus

Hardware

- Computer / Laptop

Software

- Python 3.x
- NumPy

Algorithm

Naïve Bayes Algorithm (Gaussian)

1. Separate dataset by class.
2. Compute mean and variance for each feature per class.
3. Calculate prior probabilities for each class.
4. Compute likelihood using Gaussian probability density function.
5. Predict class with maximum posterior probability.

Code

Naïve Bayes Classifier (From Scratch)

```
import numpy as np

# PSEUDOCODE:
# 1. Separate data by class
# 2. Compute mean and variance for each feature per class
# 3. Calculate prior probabilities
# 4. For each test sample:
#    a. Compute likelihood using Gaussian PDF
#    b. Multiply with prior to get posterior
#    c. Choose class with highest posterior

# Dataset
X = np.array([
    [1.0, 2.1],
    [1.5, 1.8],
    [2.0, 2.5],
    [5.0, 6.0],
    [6.0, 5.5],
    [5.5, 6.5]
])

y = np.array([0, 0, 0, 1, 1, 1])

# Separate by class
classes = np.unique(y)
mean = {}
var = {}
prior = {}

for c in classes:
    X_c = X[y == c]
    mean[c] = np.mean(X_c, axis=0)
    var[c] = np.var(X_c, axis=0)
    prior[c] = X_c.shape[0] / X.shape[0]

# Gaussian Probability Density Function
def gaussian_pdf(x, mean, var):
    exponent = np.exp(- (x - mean) ** 2 / (2 * var))
```

```
    return (1 / np.sqrt(2 * np.pi * var)) * exponent

# Prediction
def predict(X):
    predictions = []
    for x in X:
        posteriors = []
        for c in classes:
            likelihood = np.prod(gaussian_pdf(x, mean[c], var[c]))
            posterior = likelihood * prior[c]
            posteriors.append(posterior)
        predictions.append(classes[np.argmax(posteriors)])
    return predictions

# Output
y_pred = predict(X)
accuracy = np.mean(y_pred == y)
print("Classification Accuracy:", accuracy)
```

Results

- The Naïve Bayes classifier correctly classified most of the data samples.
- High accuracy was achieved for normally distributed features.
- Performance depends on the validity of independence assumption.

Discussion

Naïve Bayes is simple, fast, and effective for probabilistic classification. Although it assumes feature independence, it performs surprisingly well in many applications. Implementing it from scratch enhances understanding of probability estimation and Bayesian inference.

References

Experiment No.: 04

Experiment Name

Design and Implementation of Single-Layer Perceptron and Multi-Layer Perceptron (MLP)

Theory

Artificial Neural Networks (ANNs) are inspired by the biological nervous system. A **Perceptron** is the simplest neural network model used for binary classification.

Single-Layer Perceptron

A single-layer perceptron computes a weighted sum of inputs followed by an activation function:

$$y = f(\sum w_i x_i + b)$$

It can only solve **linearly separable** problems.

Multi-Layer Perceptron (MLP)

An MLP consists of one or more hidden layers and uses **backpropagation** to minimize error. The presence of hidden layers enables MLPs to solve **non-linear problems**.

Required Apparatus

Hardware

- Computer / Laptop

Software

- Python 3.x
- NumPy

ALGORITHM

Single-Layer Perceptron Algorithm

1. Initialize weights and bias.
2. Compute weighted sum.
3. Apply step activation function.
4. Update weights using perceptron learning rule.
5. Repeat until convergence.

MLP Algorithm (One Hidden Layer)

1. Initialize weights and biases.
2. Perform forward propagation.
3. Compute error.
4. Perform backward propagation.
5. Update weights.
6. Repeat for fixed epochs.

CODE

Single-Layer Perceptron (From Scratch)

```
import numpy as np

# PSEUDOCODE:
# 1. Initialize weights and bias
# 2. For each epoch:
#    a. Compute weighted sum
#    b. Apply step activation
#    c. Update weights if prediction is wrong

# Dataset (AND logic gate)
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0, 0, 0, 1])

# Initialize parameters
w = np.zeros(X.shape[1])
b = 0
learning_rate = 0.1
epochs = 10

# Step function
def step(x):
    return 1 if x >= 0 else 0
```

```
# Training loop
for _ in range(epochs):
    for i in range(len(X)):
        z = np.dot(X[i], w) + b
        y_pred = step(z)
        error = y[i] - y_pred
        w += learning_rate * error * X[i]
        b += learning_rate * error

print("Final Weights:", w)
print("Final Bias:", b)
```

Multi-Layer Perceptron (One Hidden Layer – From Scratch)

```
import numpy as np

# PSEUDOCODE:
# 1. Initialize weights and biases
# 2. Forward propagation using sigmoid
# 3. Compute error
# 4. Backpropagate error
# 5. Update weights and biases

# Sigmoid activation
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Dataset (XOR problem)
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

# Initialize parameters
np.random.seed(1)
W1 = np.random.rand(2, 2)
b1 = np.random.rand(1, 2)
W2 = np.random.rand(2, 1)
b2 = np.random.rand(1, 1)

learning_rate = 0.5
epochs = 5000

# Training loop
for _ in range(epochs):
    # Forward pass
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    final_output = sigmoid(final_input)
```

```
    # Error calculation
    error = y - final_output

    # Backpropagation
    d_output = error * sigmoid_derivative(final_output)
    d_hidden = np.dot(d_output, W2.T) *
sigmoid_derivative(hidden_output)

    # Update weights and biases
    W2 += learning_rate * np.dot(hidden_output.T, d_output)
    b2 += learning_rate * np.sum(d_output, axis=0, keepdims=True)
    W1 += learning_rate * np.dot(X.T, d_hidden)
    b1 += learning_rate * np.sum(d_hidden, axis=0, keepdims=True)

print("MLP Training Completed")
```

Results

- Single-Layer Perceptron successfully learned the AND gate.
- MLP successfully learned the XOR problem.
- Hidden layers enabled non-linear decision boundaries.

Discussion

Single-layer perceptrons are limited to linearly separable problems. MLPs overcome this limitation using hidden layers and backpropagation. Implementing these networks from scratch improves understanding of neural computation and learning dynamics.

References

Experiment No.: 05

Experiment Name

Development of Neuro-Fuzzy Systems for Intelligent Decision Making

Theory

A Neuro-Fuzzy System combines the learning capability of **Artificial Neural Networks (ANNs)** with the reasoning capability of **Fuzzy Logic**. Fuzzy logic handles uncertainty and imprecision using linguistic variables and fuzzy rules, while neural networks provide adaptive learning through parameter optimization.

A commonly used Neuro-Fuzzy model is the **Adaptive Neuro-Fuzzy Inference System (ANFIS)**, which consists of the following layers:

1. Fuzzification layer
2. Rule layer
3. Normalization layer
4. Defuzzification layer
5. Output layer

In this experiment, a simplified Neuro-Fuzzy model is implemented from scratch using Gaussian membership functions and gradient-based parameter updates.

Required Apparatus

Hardware

- Computer / Laptop

Software

- Python 3.x
- NumPy

ALGORITHM

Neuro-Fuzzy Algorithm

1. Define fuzzy membership functions for inputs.
2. Generate fuzzy rules.
3. Compute firing strength of each rule.
4. Normalize firing strengths.
5. Compute output using weighted sum.
6. Update parameters using learning rule.

CODE

Neuro-Fuzzy System (From Scratch)

```
import numpy as np

# PSEUDOCODE:
# 1. Initialize membership function parameters
# 2. Compute membership values for inputs
# 3. Calculate rule firing strengths
# 4. Normalize firing strengths
# 5. Compute output
# 6. Update parameters

# Gaussian membership function
def gaussian(x, mean, sigma):
    return np.exp(-((x - mean) ** 2) / (2 * sigma ** 2))

# Dataset
X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

# Initialize parameters
mean1, sigma1 = 2.0, 1.0
mean2, sigma2 = 4.0, 1.0
w1, w2 = 0.5, 0.5
learning_rate = 0.01
epochs = 1000

# Training loop
for _ in range(epochs):
    for i in range(len(X)):
        mu1 = gaussian(X[i], mean1, sigma1)
        mu2 = gaussian(X[i], mean2, sigma2)

        firing1 = mu1
        firing2 = mu2
        total = firing1 + firing2

        y_pred = (firing1 * w1 + firing2 * w2) / total
```

```
        error = y[i] - y_pred

        w1 += learning_rate * error * firing1
        w2 += learning_rate * error * firing2

print("Training completed")
```

RESULTS

- The Neuro-Fuzzy system learned the input–output relationship.
- Fuzzy rules were adjusted through learning.
- The system handled uncertainty effectively.

DISCUSSION

Neuro-Fuzzy systems integrate the strengths of neural networks and fuzzy logic. They provide interpretability through fuzzy rules while maintaining adaptability through learning. Implementing the model from scratch helps in understanding rule evaluation and parameter tuning.

REFERENCES

Experiment No.: 06

Experiment Name

Optimization of Neural Network Parameters Using Genetic Algorithms (Neuro-Genetic Approach)

Theory

Neuro-Genetic systems combine **Neural Networks (NNs)** with **Genetic Algorithms (GAs)** for optimization. Neural networks are powerful for learning complex mappings, while GAs optimize parameters (weights and biases) without requiring gradient information.

Key steps in a Neuro-Genetic system:

1. Encode NN weights and biases as chromosomes.
2. Initialize a population of chromosomes.
3. Evaluate fitness using NN performance (e.g., MSE).
4. Apply genetic operators: selection, crossover, mutation.
5. Repeat until convergence.
6. Use optimized weights in the neural network.

This approach is useful for **non-differentiable problems** or networks prone to local minima.

Required Apparatus

Hardware

- Computer / Laptop

Software

- Python 3.x
- NumPy

Algorithm

Neuro-Genetic Algorithm

1. Encode NN weights as a 1D array (chromosome).
2. Initialize a population of chromosomes randomly.
3. For each generation:
   a. Decode chromosomes into NN weights.

b. Compute fitness (e.g., MSE on training data).
c. Select parents based on fitness.
d. Apply crossover and mutation to create offspring.

4. Replace old population with offspring.
5. Repeat until maximum generations.
6. Assign best chromosome to NN weights.

CODE

Neuro-Genetic Optimization (From Scratch)

```
import numpy as np

# PSEUDOCODE:
# 1. Initialize population of random chromosomes (NN weights)
# 2. For each generation:
#    a. Evaluate fitness (MSE)
#    b. Select parents
#    c. Apply crossover and mutation
# 3. Replace population with new offspring
# 4. Return best chromosome as optimized NN weights

# Dataset (simple function approximation)
X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

# Parameters
pop_size = 10
chrom_length = 2  # weight and bias
generations = 50
mutation_rate = 0.1
learning_rate = 0.01

# Initialize population
population = np.random.randn(pop_size, chrom_length)

def nn_predict(chrom, x):
    w, b = chrom
    return w * x + b

def fitness(chrom):
    y_pred = nn_predict(chrom, X)
    return np.mean((y - y_pred) ** 2)  # MSE

def select(pop):
    # Select two best chromosomes
    sorted_pop = sorted(pop, key=fitness)
    return sorted_pop[0], sorted_pop[1]

def crossover(parent1, parent2):
    alpha = np.random.rand()
    child = alpha * parent1 + (1 - alpha) * parent2
    return child
```

```
def mutate(chrom):
    for i in range(len(chrom)):
        if np.random.rand() < mutation_rate:
            chrom[i] += np.random.randn()
    return chrom

# Genetic Algorithm loop
for _ in range(generations):
    new_pop = []
    for _ in range(pop_size):
        p1, p2 = select(population)
        child = crossover(p1, p2)
        child = mutate(child)
        new_pop.append(child)
    population = np.array(new_pop)

# Best solution
best_chrom = select(population)[0]
print("Optimized Weights and Bias:", best_chrom)
print("Predictions:", nn_predict(best_chrom, X))
```

RESULTS

- The GA successfully optimized the NN parameters.
- Mean squared error decreased with generations.
- The network predicted outputs close to the target values.

DISCUSSION

Neuro-Genetic approaches provide **gradient-free optimization** of neural network parameters. They are robust to local minima and can optimize complex architectures. This lab demonstrates the integration of evolutionary techniques with neural learning.

REFERENCES